



개발에 필요한 최신 GitHub Copilot 기능 및 유용한 팁

디모이

GitHub Dev Days SEOUL by .NET Dev

오늘의 목표

- Copilot을 **그냥 자동완성 도구**가 아닌 **에이전트**로 활용하기
- **9가지 실전 팁**으로 생산성과 정확도 동시에 높이기
- 프리미엄 리퀘스트를 **전략적으로 절약**하는 방법 익히기
- 직접 따라하는 **실습 2회** 포함

디모이 — 닷넷데브 운영진, Microsoft MVP

오늘의 아젠다

1. GitHub Copilot 2026 에코시스템
2. 커스터마이징 — Instructions, Prompts, Skills
3. 에이전틱 워크플로우 — Agents, MCP, CLI, SDK, Hooks
4. 비용 최적화 — TaskSync, 병렬 세션
5. 실습: "이해도" 퍼센테이지 기법
6. 베스트 프랙티스 & 체크리스트

GitHub Copilot in 2026



Copilot 에코시스템 한눈에

도구	역할	어디서?
Chat (Ask / Agent)	대화형 코딩 지원 (Edit 모드 Deprecated)	VS Code IDE
Agent Mode	자율적 멀티파일 편집 + 터미널	VS Code IDE
Copilot CLI	터미널 전용 AI 에이전트	터미널 / VS Code
Copilot SDK	CLI 위에 앱을 만드는 프로그래밍 API	TS·Py·.NET·Go·Java
Coding Agent	클라우드에서 자동 PR 생성	GitHub.com
Code Review	AI 기반 코드 리뷰	GitHub.com / IDE

오늘은 **VS Code + CLI + SDK** 중심으로 살펴봅니다

2026 모델 라인업 & 프리미엄 비용

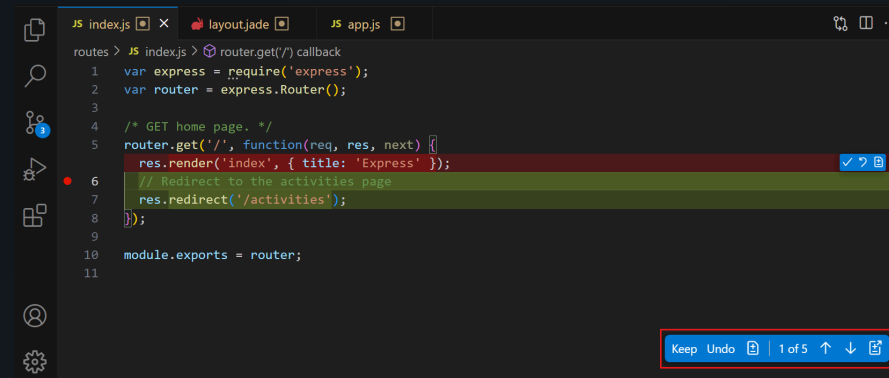
모델	멀티플라이어	특징
GPT-4.1 / GPT-4o / GPT-5 mini / Raptor mini	0x	유료 플랜 포함 (무료!)
Grok Code Fast 1	0.25x	초저가
Claude Haiku 4.5 / Gemini 3 Flash / GPT-5.4 mini	0.33x	경량 · 빠름
Claude Sonnet 4.6 / GPT-5.4 / Gemini 3.1 Pro	1x	가성비 프리미엄
GPT-5.3-Codex / GPT-5.2-Codex	1x	코딩 특화
Claude Opus 4.6	3x	최고 성능

Auto Model Selection 사용 시 유료 플랜 **10% 할인** · BYOK로 외부 모델도 연결 가능

Agent Mode — AI가 직접 코딩

- **자율 판단**: 어떤 파일을 수정할지 스스로 결정
- **멀티스텝**: 파일 편집 → 터미널 실행 → 에러 수정 반복
- **MCP 통합**: 외부 도구·API를 에이전트가 직접 호출
- **도구 승인**: 각 단계에서 사용자 확인 또는 자동 승인 선택

Chat 뷰 상단에서 Agent 모드 선택 → 프롬프트 입력 → 끝!



```
routes > JS index.js > router.get('/') callback
1  var express = require('express');
2  var router = express.Router();
3
4  /* GET home page. */
5  router.get('/', function(req, res, next) {
6    res.render('index', { title: 'Express' });
7    // Redirect to the activities page
8    res.redirect('/activities');
9  });
10
11 module.exports = router;
```

Copilot CLI — 터미널의 AI 에이전트

- **2가지 모드**: 대화형 (`copilot`) · 헤드리스 (`copilot -p "프롬프트"`)
- **Agent 모드**: 파일 편집, Git 커밋, 테스트 실행까지 자율 수행
- **MCP 서버 연결**: 외부 데이터 소스 · GitHub API 등 접근 가능
- **커스텀 에이전트**: 작업별 전문 페르소나 자동 위임
- **Plan 모드** (`Shift+Tab`): 실행 전 계획 미리보기
- VS Code 안에서 **백그라운드 에이전트**로도 작동

```
$ copilot  
> 이 프로젝트의 빌드 에러를 찾아서 수정 해줘
```

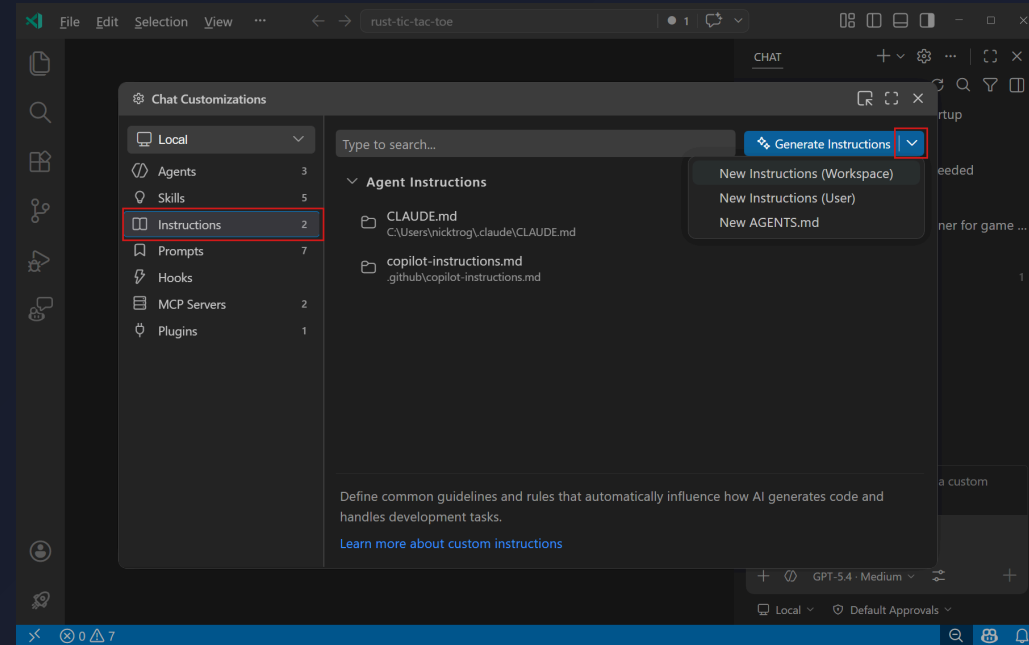

Tip 1: 세션 분기(Fork)로 마음껏 탐색하기

- "이 방향이 맞나?" 고민될 때 → `/fork` 으로 현재 대화를 **통째로 복제**
- 원본 세션은 **그대로 보존**, 새 세션에서 자유롭게 실험
- **체크포인트 분기**: 원하는 시점에 마우스 hover → **Fork Conversation** 클릭
 - 그 시점까지의 대화만 가져와서 **새로운 방향** 탐색

Chat에서 `/fork` 입력 → 전체 대화 복제 ("Forked:" 접두사)
체크포인트 hover → Fork 버튼 → 특정 시점까지만 복제

되돌리기가 두렵다면 **분기(Fork)!** 원본 손실 없이 여러 접근법을 동시에 탐색

프로젝트 커스터마이징



Tip 2: `/init` 한 방으로 Instructions 자동 생성

- `/init` 입력 → 프로젝트를 분석해서 **copilot-instructions.md** 자동 생성
- 직접 처음부터 쓸 필요 없음! AI가 스탭.컨벤션을 파악해서 초안 작성

실전 꿀팁

- **파일별 규칙 분리**: `applyTo: '**/*.py'` 로 Python만, `applyTo: 'tests/**'` 로 테스트만 적용
- **짧게 유지**: 매 요청마다 로드되므로 코드에서 유추 가능한 당연한 규칙은 빼기
- **"왜"를 적기**: `moment.js` 대신 `date-fns` → "번들 크기 작고 moment는 deprecated"
- **대화 중 규칙 추출**: `/create-instruction` → AI가 대화에서 패턴을 발견해 자동 생성

`/instructions` → Instructions 설정 메뉴 열기 · `/create-instruction` 으로 대화 중 패턴을 자동 추출 가능

Tip 3: Prompt File 만드는 2가지 방법

- `/create-prompt` + 작업 설명 → AI가 질문 후 `.prompt.md` 새로 생성
- `/savePrompt` → 현재 대화를 분석해 **템플릿화된** `.prompt.md` 자동 저장
- 대화 도중 "이걸 프롬프트로 저장해줘"라고 요청해도 OK

프론트매터로 동작 제어

- `${input:변수}`: 실행 시 값을 물어봄 → 동적 템플릿
- `model`: 최적 모델 고정 / `tools`: 필요 도구만 지정

```
# .github/prompts/api.prompt.md
description: "REST API 생성"
model: Claude Sonnet 4.6
tools: ["web/fetch", "read_file"]
```

Tip 4: description이 스킬 자동 로드와 핵심

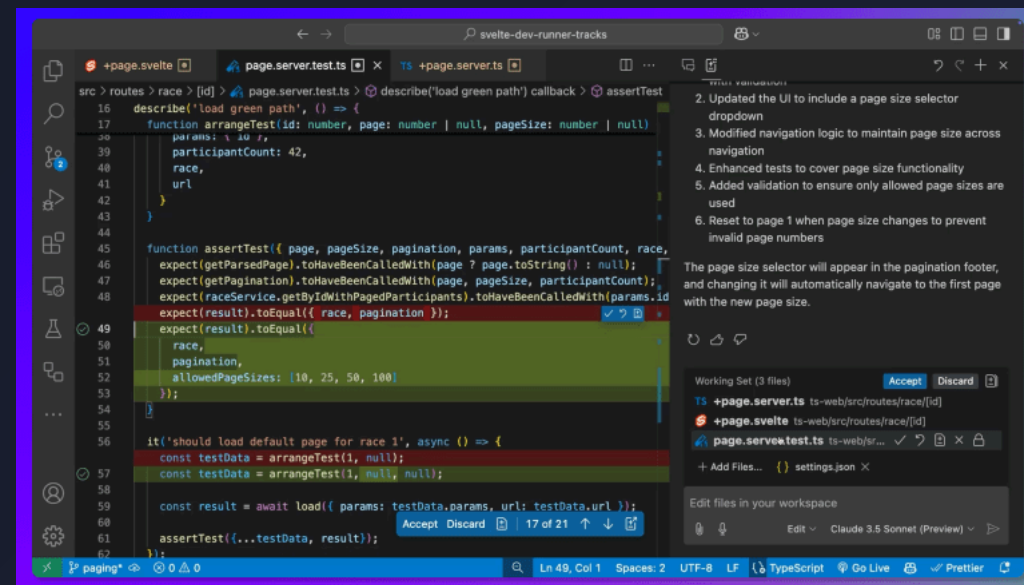
- Agent가 요청 → 모든 SKILL.md의 description 스캔 → 관련 스킬 자동 활성화
- description을 구체적이고 키워드 풍부하게 쓸수록 매칭 정확도 상승

실전 꿀팁

- "이걸 스킬로 만들어줘" → AI가 SKILL.md + 참고 파일 자동 생성
- description 예시:
 - Bad: "테스트 관련 스킬" → 모호해서 엉뚱한 작업에도 로드
 - Good: "Vitest 단위 테스트. React 컴포넌트, API 핸들러 커버리지"
- 참고 파일을 스킬 폴더에 함께 넣으면 일관된 출력 보장

잘 만든 스킬 = 전문 지식의 자동화 · 팀원 온보딩에도 효과적!

에이전틱 워크플로우



Tip 5: Custom Agents

역할별 AI 페르소나 — `.github/agents/<이름>.agent.md`

```
---
name: "코드 리뷰어"
description: "코드 품질과 보안을 검토합니다"
tools: ["read_file", "grep_search"] # 읽기 전용으로 제한
model: Claude Sonnet 4.5           # 모델 지정 가능
handoffs:
  - label: "구현 시작"              # 워크플로우 전환 버튼
    agent: implementation
---
```

OWASP Top 10 · 성능 병목 · 코딩 컨벤션을 검토하세요.

- Chat 에이전트 드롭다운에서 **선택** 또는 `/create-agent` 로 생성
- **Handoff**: 리뷰 → 구현처럼 **에이전트 간 워크플로우 전환** 지원
- 조직 수준 공유 가능 (GitHub Organization Custom Agents)

Custom Agent 만들어 보기

1. `/create-agent` 보안 리뷰 전문 에이전트 →
AI가 질문 후 자동 생성

2. 또는 직접 파일 생성:

```
# .github/agents/security-reviewer.agent.md
---
name: "보안 리뷰어"
description: "OWASP Top 10 기준 보안 취약점 검토"
tools: ["read_file", "grep_search"]
model: Claude Sonnet 4.6
handoffs:
  - label: "취약점 수정 시작"
    agent: implementation
    send: false
---
```

3. Chat 에이전트 드롭다운 → "보안 리뷰어" 선택
Handoff 흐름:

- 리뷰 완료 → "취약점 수정 시작" 버튼 표시
- `send: false` → 수동 확인 후 전송
- 클릭 → 구현 에이전트로 **자동 전환**

핵심 필드:

- `tools` — 사용 가능 도구 제한 (읽기 전용 등)
- `model` — 에이전트별 최적 모델 지정
- `handoffs` — 에이전트 간 워크플로우 연결

Tip 6: Memory에 맡길 것 vs Instructions에 넣을 것

- **Memory**: Copilot이 알아서 기억 · 28일 후 자동 삭제 · 리포별 누적
- **Instructions**: 영구 · 버전 관리 · 팀 공유 가능

Memory에 맡기기	Instructions에 넣기
"이 프로젝트는 PostgreSQL 사용"	코딩 컨벤션, 아키텍처 패턴
"디플로이는 Azure App Service"	린트 규칙, 네이밍 규칙
개인적 선호·습관	팀 공유 필수 사항

- CLI · Coding Agent · Code Review가 **메모리를 공유** (IDE 확장 예정)
- Settings → Copilot → **Memory 관리** 가능 (삭제/확인)

임시적 맥락은 Memory, 영구 규칙은 Instructions → **반복 설명 제로!**

Tip 7: MCP는 많이 연결할수록 느려진다

- Extensions 뷰에서 `@mcp` 검색 → 갤러리에서 **원클릭 설치**
- 활성 도구가 적을수록 **응답 속도↑ 정확도↑**

실전 꿀팁

- **필요한 MCP만** 활성화, 나머지 비활성화
- **HTTP** (원격: GitHub MCP) vs **stdio** (로컬: Playwright) 선택
- **MCP Apps**: 폼·차트 같은 **인터랙티브 UI**를 Chat 안에 렌더

```
// .vscode/mcp.json
{ "servers": {
  "github": { "type": "http", "url": "https://api.githubcopilot.com/mcp" },
  "playwright": { "command": "npx", "args": ["-y", "@microsoft/mcp-server-playwright"] }
} }
```

Tip 8: CLI Headless 모드로 CI/CD 자동화

```
# 터미널에서 AI 에이전트 자동 실행 (대화형 아님!)  
$ copilot -p "이 프로젝트에 ESLint + Prettier 설정 추가해줘"
```

실전 꿀팁

- **Shift+Tab** (Plan 모드): 실행 전 **계획 미리보기** → 확인 후 실행
- **/compact**: 긴 대화 컨텍스트 압축 → **토큰 절약**
- VS Code에서 **백그라운드 에이전트**로 실행 → 편집기 작업과 **병렬 수행**
- **/yolo** — 모든 도구 호출 **자동 승인** 토글
- **/context** — 토큰 사용량 상세 확인

IDE 밖에서도 Copilot = **CI/CD 파이프라인의 AI 에이전트**

Copilot SDK — CLI 위에 앱을 만드는 도구

CLI 엔진을 프로그래밍 방식으로 제어 (TypeScript · Python · .NET · Go · Java)

CLI vs SDK

	CLI	SDK
용도	터미널 대화형 작업	앱에 AI 에이전트 내장
실행	<code>copilot</code> 명령	코드에서 API 호출
자동화	Headless (<code>-p</code>)	CI/CD · 배치 처리

.NET 예시

```
using GitHub.Copilot.SDK;

var client = new CopilotClient();
await client.StartAsync();

var session = await client
    .CreateSessionAsync(
        new SessionConfig {
            Model = "gpt-5.4"
        });
```

Tip 9: Hook 하나로 AI 코드 품질 보장

필수 Hook: AI가 파일 수정할 때마다 자동 포매팅

```
// .github/hooks/format.json
{ "hooks": {
  "PostToolUse": [{
    "type": "command",
    "command": "npx prettier --write \"$TOOL_INPUT_FILE_PATH\""
  }]
} }
```

실전 꿀팁

- **PreToolUse:** `rm -rf` 같은 위험 명령 자동 차단
- **Stop:** 세션 종료 시 "테스트부터 돌려" → 자동 품질 검증
- `/create-hook` → AI가 **Hook** 파일 자동 생성

Hook 만들어 보기

1. `/create-hook` AI가 파일 수정 후 자동 포매팅 → 자동 생성 · 또는 직접 생성:

2. 자동 포매팅 — `.github/hooks/format.json`

```
{ "hooks": {  
  "PostToolUse": [{  
    "type": "command",  
    "command": "npx prettier --write \"$TOOL_INPUT_FILE_PATH\""  
  }]  
} }
```

3. 위험 명령 차단 —

`.github/hooks/safety.json`

```
{ "hooks": {  
  "PreToolUse": [{  
    "type": "command",  
    "command": "echo '$TOOL_INPUT' | grep -qE 'rm -rf|drop table' && exit 2 || exit 0"  
  }]  
} }
```

PostToolUse = 수정 후 자동 실행 · **PreToolUse** = 실행 전 검증/차단 · exit 2 = 차단, exit 0 = 허용

Claude Code vs GitHub Copilot 대응표

기능	Claude Code	GitHub Copilot
프로젝트 규칙	CLAUDE.md	copilot-instructions.md
파일별 규칙	.claude/rules/	.instructions.md (applyTo:)
재사용 프롬프트	Custom Commands	Prompt Files (.prompt.md)
스킬 패키지	SKILL.md	SKILL.md (호환!)
커스텀 에이전트	.claude/agents/*.md	.github/agents/*.agent.md
자동 메모리	Auto Memory	Copilot Memory (28일)
MCP 연동	settings.json	.vscode/mcp.json
훅/플러그인	Hooks · Plugins	Hooks · Extensions

Agent Skills 오픈 스탠더드 준수 → SKILL.md 서로 호환!

비용 최적화: TaskSync

TaskSync란? — `ask_user`로 프리미엄 극대화

- Agent의 자율적 도구 호출은 **프리미엄 리퀘스트 차감 안 됨** (무료!)
- 문제: Agent가 작업 완료 후 **세션 종료** → 다음 작업엔 새 리퀘스트 필요
- **TaskSync**: `ask_user` 톨로 "다음 할 일?" 질문 → 자동 응답 → **세션 유지!**

방식	작업량	프리미엄 사용
작업마다 새 세션	5개	5회
TaskSync (1세션 유지)	5개	1회

1 프리미엄 리퀘스트로 **5배, 10배** 작업 가능!

TaskSync 따라해 보세요!

1. 확장 프로그램 설치: VS Code 마켓플레이스에서 **TaskSync** 검색 설치

2. Custom Instructions에 **TaskSync 지시문** 추가:

모든 작업 후 반드시 `ask_user` 도구를 호출하여 피드백을 요청하세요.
사용자가 명시적으로 "종료"라고 할 때까지 `ask_user` 호출을 멈추지 마세요.

3. TaskSync 패널 → **Queue Mode** ON → 프롬프트 미리 추가:

- "단위 테스트 작성해줘"
- "README.md 업데이트해줘"
- "빌드 스크립트 최적화해줘"

4. Agent에게 첫 작업 요청 → 완료 후 **자동으로 다음 큐 전송!**

병렬 세션 & Subagents — AI 팀워크

병렬 세션: Chat + 버튼으로 독립 작업을 동시에 (서로 다른 파일 권장)

Subagents — 한 프롬프트로 AI 팀 가동 (Experimental)

- 메인 Agent가 **서브에이전트를 자동 호출** → 결과만 돌려받음
- 각 서브에이전트는 **독립 컨텍스트**에서 작업 → 간섭 없음
- Custom Agent를 서브에이전트로 지정하면 **모델·도구·지시사항 분리**

Coordinator (Feature Builder)	역할
① Planner (read-only)	작업 분해
② Implementer (edit)	코드 구현
③ Reviewer (read-only)	코드 리뷰
④ 이슈 발견 시	Implementer 재호출

Subagents 만들어 보기

1. Coordinator — `agents` 로 서브에이전트 제한

```
---
name: TDD
tools: ['agent', 'edit', 'search', 'read']
agents: ['Red', 'Green', 'Refactor']
---
```

1. Red → 실패 테스트 작성
2. Green → 테스트 통과 코드 구현
3. Refactor → 코드 개선

2. Worker — `user-invocable: false` 로 전용

```
---
name: Red
user-invocable: false
tools: ['read', 'search', 'edit']
---
```

실패하는 테스트만 작성하세요.

3. 설정: `chat.subagents.allowInvocationsFromSubagents` → 중첩 허용 (최대 5단계)

"이해도" 퍼센테이지 기법

ot



opilot

.Net
Dev

Before → After: 이해도 기법

Before

앱 만들어줘

→ Copilot이 **추측**해서 아무거나 만들

→ 재작업 = **시간 + 프리미엄 낭비**

After

앱 만들어줘 .

이해도를 **0~100%**로 표시하고,
95% 미만이면 선택형 질문을
제시해줘 .

→ **이해도: 25%** + 질문 5개 제시

→ 답변 → **95%** → 정확한 결과!

프롬프트 마지막에 3줄만 추가하면 Copilot이 **먼저 질문**하고, **정확하게** 작업합니다

이해도 프롬프트 템플릿

[당신의 요청 내용]

위 요청에 대한 이해도를 0~100%로 표시해줘.

95% 미만이면 이해도를 높이기 위한 선택형 질문을 3~5개 제시해줘.

활용 꿀팁

- 이 프롬프트를 **Prompt File** (`.prompt.md`)로 저장 → `/이해도` 로 재사용
- **Custom Instructions**에 넣으면 모든 대화에 자동 적용
- 95% 이상 달성 후 작업 시작하면 **한 번에 원하는 결과!**

실습: 이해도 기법 직접 적용

Step 1. VS Code Chat 열기 → 아래 프롬프트 입력:

로그인 기능 만들어줘.
이 요청의 이해도를 0~100%로 표시하고,
95% 미만이면 선택형 질문을 제시해줘.

Step 2. Copilot의 응답 확인 → 이해도 30~50% + 질문 3~5개

Step 3. 제시된 질문에 구체적으로 답변 (React + Next.js, OAuth 2.0 등)

Step 4. 이해도 95% 이상 달성 확인! → Before vs After 차이를 직접 체감!

베스트 프랙티스

ot



opilot



프리미엄 리퀘스트 아끼는 꿀팁

전략	효과
Auto Model Selection 사용	10% 할인 + 최적 모델 자동 선택
간단한 작업엔 포함 모델 (GPT-4.1 등)	0x (무료!)
새 주제 = 새 세션 : 컨텍스트 오염 방지	정확도 향상
<code>/compact</code> 로 긴 대화 압축	토큰 절약
프롬프트에 테스트 케이스 함께 전달	AI 자가 검증
이해도 기법 으로 첫 프롬프트부터 정확	재작업 감소
TaskSync 로 1세션에서 연속 작업	프리미엄 5~10배 절약

오늘 바로 해볼 체크리스트

- [] **Tip 1** `/fork` 로 세션 분기 탐색
- [] **Tip 2** `/init` 으로 Instructions 생성
- [] **Tip 3** `/create-prompt` 로 Prompt File
- [] **Tip 4** 반복 작업을 Skill로 패키지화
- [] **Tip 5** `/create-agent` 로 Custom Agent
- [] **Tip 6** Memory vs Instructions 구분
- [] **Tip 7** `@mcp` 로 필요한 MCP만 연결
- [] **Tip 8** CLI Headless로 자동화
- [] **Tip 9** `/create-hook` 으로 Hook 설정
- [] **+** 이해도 프롬프트를 Prompt File 저장
- [] **+** TaskSync 설치 + 큐 모드 활용

더 알아보기 최신 기능 (Preview 포함)

GA (정식 출시)	핵심
NES	Tab 으로 다음 수정 위치 자동 이동
Vision	스크린샷·목업을 이미지로 첨부
Thinking Effort	추론 강도 Low / Medium / High
Coding Agent	50% 빠른 시작 · 머지 충돌 해결

Preview / Experimental	핵심
Agent Plugins	스킬+에이전트+혹+MCP 번들
Customizations Editor	커스터마이징 통합 관리 UI
Agent Debug Logs	에이전트 이벤트 로그 디버깅

Edit 모드 → **Deprecated** (Ask + Agent로 통합)

Q&A

질문 있으신가요?

발표 자료와 데모 코드는 추후 공유할 예정입니다

감사합니다!

GitHub Copilot
Dev Days

SEOUL April 3



by .NET Dev

